

SPRING BOOT BASICS

What is Spring Boot in very simple words?

Spring Boot is a tool that makes the original Spring Framework incredibly easy to use. It simplifies the setup process by providing sensible default configurations. Essentially, it allows you to quickly build and run a production-ready Java application with minimal effort.

What basic features make Spring Boot easier than normal Spring?

The main features are **Auto-Configuration** and **Starter Dependencies**. Auto-Configuration automatically configures your application based on what it finds on the classpath (e.g., if it finds JPA, it sets up the database connection). Starter dependencies bundle common libraries together, avoiding version conflicts and manual setup.

Why do developers prefer Spring Boot instead of doing Spring setup manually?

Setting up traditional Spring manually involves writing a lot of repetitive configuration code (XML or Java files). Developers prefer Spring Boot because its defaults handle this automatically. This saves significant development time, reduces setup mistakes, and allows developers to focus immediately on writing business logic.

What are the main parts of a Spring Boot application?

The main parts are the **Application Class** (with `@SpringBootApplication`), the **Configuration Files** (application.properties), and the **Beans** (your components like services and controllers). Everything is packaged into a single, executable JAR file.

What do Spring Boot “starter” dependencies do?

Starter dependencies are handy pre-packaged sets of libraries that you can add to your project with a single line in your build file. They pull in all the necessary common dependencies and transitively manage their versions for you. This prevents dependency conflicts and ensures quick setup for features like web development or database access.

What happens inside Spring Boot when you run the application?

When you run the application, the **IoC Container** starts up. It performs **Auto-Configuration** and **Component Scanning** to find your beans and apply defaults. The embedded server (usually Tomcat) then starts, and the application becomes ready to handle incoming web requests.

What is Spring Initializr, and why do developers use it?

Spring Initializr is a web-based tool (start.spring.io) that lets you quickly generate the basic project structure and necessary build files (Maven/Gradle). Developers use it because it

handles the initial dependency management and project scaffolding automatically, allowing them to download a working skeleton project instantly.

What does the word “Spring Bean” mean in Spring Boot?

A **Spring Bean** is any Java object that is instantiated, assembled, and managed entirely by the **Spring IoC Container**. Essentially, if Spring creates the object for you, it is called a Bean. Your services, controllers, and repositories are all Beans.

What does auto-wiring mean in Spring?

Auto-wiring is the process where Spring automatically finds and connects the necessary dependency objects into a component that needs them. When you use the `@Autowired` annotation, Spring looks for a matching bean (by type or name) in the container and injects it into your class, saving you from manual object creation.

What does dependency management mean in Spring Boot?

Dependency management refers to Spring Boot's ability to automatically select and handle compatible versions for all your libraries. The parent POM (spring-boot-starter-parent) centrally manages the versions of common dependencies. This prevents the frequent problems that arise when different libraries accidentally require incompatible versions of shared components.

SERVER & CONFIGURATION BASICS

Can you change the default port of the embedded server? How do you do it?

Yes, you can easily change the default port. You do this by setting the property `server.port` in your **application.properties** or **application.yml** file. For example, setting `server.port=8081` will make the server run on port 8081 instead of the default 8080.

Which port does Spring Boot use by default?

Spring Boot uses port **8080** by default for the embedded web server.

Which built-in servers can Spring Boot use, and which one comes by default?

Spring Boot can use **Tomcat**, **Jetty**, and **Undertow**. The server that comes by default when you use the spring-boot-starter-web dependency is **Tomcat**.

What is the job of the pom.xml file in a Maven Spring Boot project?

The pom.xml file (Project Object Model) is the main configuration file for Maven. Its job is to declare the project's dependencies (libraries), manage the build process, and define project metadata like versioning and packaging instructions.

Why do we use the spring-boot-starter-parent in Maven?

We use the spring-boot-starter-parent to inherit crucial default configurations and dependency management. It provides sensible defaults for things like Java version, resource filtering, and plugin configurations. Most importantly, it centrally manages the versions of all Spring dependencies, ensuring compatibility.

How do Spring Boot starters make dependency handling easier?

Starters make dependency handling easier by bundling common technologies together. For example, spring-boot-starter-web brings in Tomcat, Spring MVC, Jackson, and more, all with tested, compatible versions. You only need to list one dependency instead of ten.

REST API BASICS

What does @RestController do?

@RestController is a convenience annotation used on a class to handle incoming web requests. It combines two other annotations: **@Controller** and **@ResponseBody**. The **@ResponseBody** part means that the data returned by the methods is automatically converted into a web format (usually JSON or XML) and sent directly in the HTTP response body.

How is @RestController different from @Controller?

@Controller is the general-purpose annotation used to define a class that handles web requests, often used in traditional web apps that return view names (HTML pages).

@RestController is specialized for building REST APIs, as it automatically includes the **@ResponseBody** behavior, removing the need to manually annotate every method to return JSON.

What is the difference between @RequestMapping and @GetMapping?

@RequestMapping is the general annotation used to map any HTTP request method (GET, POST, PUT, etc.) to a handler method. **@GetMapping** is a specialized, shortcut annotation introduced in Spring 4.3 that is strictly equivalent to **@RequestMapping(method = RequestMethod.GET)**. Developers prefer **@GetMapping** because it is clearer and more concise.

What does @RequestBody mean in a REST API?

@RequestBody is used to tell Spring to convert the incoming data from the HTTP request body (usually a JSON payload) into a specific Java object. Spring automatically handles the conversion process (deserialization) before the method is executed.

Can you use **@Component** instead of **@Service** or **@Repository**? Why do these separate annotations exist?

Yes, you *can* use **@Component** in place of **@Service** or **@Repository**, as **@Service** and **@Repository** are specialized forms of **@Component**. These separate annotations exist to improve code clarity and apply specific behavior. For example, **@Repository** automatically enables exception translation, converting low-level database errors into Spring Data exceptions.

How do you create simple CRUD APIs in Spring Boot?

You create CRUD (Create, Read, Update, Delete) APIs by defining a **Controller** class with methods mapped to the appropriate HTTP methods. These methods call a **Service** layer, which in turn uses a **Repository** (like **JpaRepository**) to interact with the database.

What do the common HTTP methods (GET, POST, PUT, PATCH, DELETE) mean?

- **GET**: Retrieve data.
- **POST**: Create new data.
- **PUT**: Fully replace/update existing data.
- **PATCH**: Apply partial modifications to existing data.
- **DELETE**: Remove data.

When should you use GET, POST, PUT, PATCH, and DELETE in REST?

You should use **GET** to fetch resources (safe and idempotent). **POST** is used to submit data and create a new resource. **PUT** is used to fully update a resource (sending the entire object). **PATCH** is used to update only a few fields on a resource. **DELETE** is used to remove a resource.

What is the correct status code to return after deleting something?

The two most correct status codes are **204 No Content** (if you return nothing in the body) or **200 OK** (if you return a message confirming the deletion). **202 Accepted** is also sometimes used if the deletion process will take some time and is running asynchronously.

Can you list five commonly used annotations for REST APIs in Spring Boot?

1. **@RestController** (Class level)
2. **@GetMapping** (Method level)
3. **@PostMapping** (Method level)
4. **@PathVariable** (Parameter level)

5. **@RequestBody** (Parameter level)

SWAGGER / API DOCUMENTATION

What is Swagger?

Swagger (now part of the OpenAPI Specification) is a framework that helps developers design, build, document, and consume REST APIs. It defines a standard, language-agnostic interface for REST APIs, allowing humans and computers to discover and understand the service's capabilities without having access to source code.

How does Swagger help in testing and documenting APIs?

Swagger automatically generates documentation from your code annotations, providing a clean, interactive user interface (Swagger UI). This interface allows developers and testers to **execute** API calls directly from the browser, which simplifies manual testing and ensures documentation always reflects the latest code.

TESTING BASICS

How can you test a Spring Boot application in simple ways?

You can test an application in three main ways: **Unit Tests** (testing single classes in isolation), **Integration Tests** (testing how multiple layers like the Service and Repository work together), and **End-to-End Tests** (testing the entire application flow via HTTP requests).

Why do we write unit tests?

We write unit tests to verify that individual pieces of code (usually single methods or classes) work exactly as intended. They are fast, help catch bugs early, and act as living documentation for what the code is supposed to do.

How do JUnit and Mockito help in testing?

JUnit is the framework used to write and run the actual tests (the test harness). **Mockito** is a powerful framework used to create **mock objects** (fake dependencies). Mockito lets you isolate the class being tested by replacing its real dependencies (like a database repository) with objects that return predictable results.

What is the difference between @Mock and @InjectMocks?

@Mock creates a completely fake object that you must tell what to do (a mock dependency). **@InjectMocks** creates the *actual instance* of the class you want to test.

Mockito then attempts to inject all the fields annotated with `@Mock` into that `@InjectMocks` instance.

What is `@SpringBootTest` used for?

`@SpringBootTest` is used for running full **integration tests**. It tells JUnit to launch the entire Spring `ApplicationContext`. This allows you to test the complete application stack, from the controller down to the repository, as if the application were actually running.

What do `@Before`, `@After`, and `@BeforeAll` mean in JUnit?

- **`@BeforeEach`**: Runs before every single test method.
- **`@AfterEach`**: Runs after every single test method.
- **`@BeforeAll`**: Runs once before any of the test methods in the class start.

How would you use these methods inside a simple test class?

You use `@BeforeEach` to set up initial conditions needed by every test, like resetting mock objects or initializing variables. You use `@AfterEach` to perform cleanup tasks, and `@BeforeAll` to initialize expensive resources, like loading a static data file, only once per class.

LOGGING & DEVTOOLS

How do you turn on debug-level logging in Spring Boot?

You turn on debug logging by setting the property `logging.level.root=DEBUG` in your **application.properties** file. You can also turn it on specifically for a single package or class (e.g., `logging.level.com.app.service=DEBUG`).

What are Spring Boot DevTools, and how do they help developers?

Spring Boot DevTools is a library that provides useful developer-only tools, primarily focused on improving workflow speed. The main feature is **automatic application restart** whenever you save code changes.

Why should you never use DevTools in production?

You should never use DevTools in production because they introduce **security risks** and **performance overhead**. Features like the H2 console and automatic restarting are designed only for a local development environment.

How do DevTools make development faster when your code changes often?

DevTools use **LiveReload** and a secondary classloader to detect code changes and trigger an immediate, fast restart of the application. This saves the developer from manually stopping and starting the server every time they make a small change to a class or configuration file.

COMMON SPRING BOOT BASICS

What basic annotations should every Spring Boot developer know?

1. `@SpringBootApplication` (Main class)
2. `@Autowired` (Dependency Injection)
3. `@RestController` (Web API)
4. `@Service` (Business Logic)
5. `@Repository` (Data Access)

Can you name a few common Spring Boot starters?

Common starters include **spring-boot-starter-web** (for web development), **spring-boot-starter-data-jpa** (for database access), and **spring-boot-starter-test** (for testing libraries).

What is `application.properties` or `application.yml` used for?

These files are used to centrally configure your application. They allow you to set simple values like the database URL, server port, logging levels, and other framework settings without changing the actual Java code.

What does `@SpringBootApplication` do?

`@SpringBootApplication` is a convenience annotation placed on the main class that combines three other annotations: `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`. It signals to Spring: "Start here, apply automatic configuration, and look for components in this package."

How does Spring Boot find and load your components automatically?

Spring Boot uses **Component Scanning** (enabled by `@ComponentScan`) to find components. It looks in the main application class's package and all sub-packages for classes annotated with stereotypes like `@Component`, `@Service`, `@Repository`, and `@Controller`, and registers them as beans.

What does `@PathVariable` do in a REST API?

@PathVariable is used to extract a dynamic value directly from the URL path. For example, if the path is `/users/{id}`, **@PathVariable** extracts the ID value (like 101) from the path and maps it to a method argument.

What is the difference between **@RequestParam** and **@PathVariable**?

@PathVariable extracts values from the actual **URL path** and is used to identify a specific resource (e.g., `/users/101`). **@RequestParam** extracts values from the **query parameters** at the end of the URL and is used for sorting, filtering, or optional data (e.g., `/users?status=active`).

JAVA Interview Buddy